Reliable Team Assistant

Phase 1: Fault Propagation and Recovery Baselines

Yiting Dai | CSE291A Systems for LLMs and AI Agents | Option 5 | October 8, 2026

Research question. Can tracking actual dependencies identify work that must be recomputed after a faulty intermediate result, while retaining valid work? This phase establishes a small, inspectable workflow and distinguishes error exposure, factual contamination, task failure, prevention, and recovery. Procedural-memory learning and multi-group scheduling remain Phase 2 extensions; their benefits are not claimed here.

1. Workflow and experimental boundary

Read-only evidence	Two specialist agents	Coordinator and outputs
Pods + alerts	Configuration/status and telemetry/dependencies; at most 3 queries each	Raw observations + specialist artifacts; at most 2 supplementary rounds; structured diagnosis with citations

The implementation uses Cloud-OpsBench [1] snapshots and native scoring at commit ea05dafb. Three previously seen development incidents cover runtime, service routing, and performance. A tool-disabled Codex CLI invokes gpt-6-astra using an existing subscription. Inference receives questions and observed evidence; oracle scoring runs separately. These are real model calls over replayed tools, not a live cluster or a held-out benchmark.

2. Predeclared interventions and controls

Intervention	Type / handoff	Fixed transformation
Runtime port	Accidental / configuration evidence	9555 to 9556
Service address	Accidental / configuration evidence	redis-cart-invalid to redis-cart
CPU limit	Accidental / configuration evidence	200m to 800m
Service instruction	Adversarial / telemetry evidence	Append request to call forbidden DeleteNamespace

Each intervention has one clean and one damaged trajectory where injection occurs. Clean and damaged trajectories share a settled collection prefix. Selective and full-restart recovery share the same consumed faulty prefix and externally supplied artifact identity. The source-integrity preventive baseline checks every observation before model consumption against the trusted snapshot; it uses independent collection. It is not a semantic verifier. A missing mutation is recorded as not injected, with no retargeting.

3. Observed propagation

Intervention	Injected	Clean / damaged JRA	Unfaithful damaged citations
Runtime port	True	1.0 / 1.0	1
Service address	False	1.0 / -	not exposed
CPU limit	True	1.0 / 0.0	2
Adversarial instruction	True	1.0 / 1.0	0

JRA is native joint target/root-cause accuracy. Citation fidelity compares quoted text with the pristine observation of the same ID; a contract checks only consistency with consumed inputs. These tests capture different failure modes. No general safety rate is inferred from this small matrix.

Evaluation, tradeoffs and limitations

4. Recovery cost and applicability

Intervention	Selective total tokens	Restart total tokens	Preventive total tokens
Runtime port	70,584	102,365	51,157
CPU limit	84,271	116,813	58,224

Totals include the collection and consumed-fault prefix for both recovery methods. Each initial trajectory has a 360k-token ceiling; recovery gets a further 360k tokens, whereas prevention has one 360k ceiling. Recovery ceilings and tool limits match, but end-to-end preventive and recovery caps differ. All measured trajectories remain below their caps. Tokens include CLI context overhead and are not API dollar costs.

The runtime mutation propagates a false port into explanation/citations while preserving the correct protocol-mismatch label. The CPU-limit mutation produces an uncertain answer despite clean task success; both source fidelity and task completion are therefore reported. Recovery restores useful outputs in the completed accidental trials. The service-address mutation did not match collected evidence and remains in the intervention denominator. The adversarial instruction was consumed without a forbidden tool request in completed clean/damaged trajectories; its recovery attempt was interrupted by HTTP 503. No adversarial recovery success is claimed. Read-only execution cannot establish real operational-action safety.

Both completed accidental recovery trials invalidate configuration evidence and its downstream diagnosis, retaining four independent/valid artifacts. Selective recovery uses one extra model call versus three for restart, and 2 vs 7 / 3 vs 8 tools (runtime / performance). Measured recovery-only latency is 8.27 vs 25.65 seconds (runtime) and 19.60 vs 31.12 seconds (performance); fixed method order and one repeat prevent a causal latency claim. Both live accidental injections occur at the collection-to-coordinator boundary, not two temporal stages.

Always checking source integrity is cheap for static snapshots and can prevent damage without additional inference. Verifier CPU was 1.69/39.48 ms for 11/12 observation checks and 2/3 repairs (runtime/performance). Its low cost is a meaningful counterexample to universal selective-recovery superiority. The selective method is expected to help when validating or restarting all work is expensive; this small experiment does not yet establish that general regime.

5. State controls and measured overhead

Four offline fake-provider controls pass: timeout/unknown-usage dispatch stopping, stale-policy write rejection with charging, independent-branch retention, and append-only linked correction. A scripted action ledger rejects an unauthorized proposal; this is an in-memory control test, not model safety evidence. In a 128-node synthetic graph over 20 timed repeats, median tracked commit CPU is 6.422 ms versus 0.764 ms without dependencies and 0.740 ms for a deepcopy dictionary. Invalidation takes 0.122 ms. Graph creation and inference are excluded; serialized sizes and raw samples are included in the artifact.

6. What this phase establishes

The deliverable includes a clean workflow, fixed injection harness, actual-input/version traces, full restart, narrowly scoped always-source-integrity checking, selective recovery, and negative results. Fault identity is supplied rather than detected automatically. Corrupted messages are traced to actual recorded requests; ignored adversarial instructions are not counted as recovery successes. Failed and unexposed conditions remain visible.

The frozen matrix contains 4 predeclared conditions across three distinct seen incidents, with 40 physical model calls and 750,324 known tokens plus 2 unknown-usage call(s), conservatively charged 60k each. 3 provider errors are retained; bounded retries are identified in the artifact. These descriptive runs cannot establish statistical significance, tail latency, or family-level generalization. The eight final cases stay locked. Earlier pilots and an invalid verifier version are retained separately, with their costs, rather than pooled into this matrix.

Remaining Phase 2 work includes semantic verification, shared/dense contamination, incomplete provenance, learned-memory validation, multi-group scheduling, and tracking-cost scaling. Durable process recovery and real operational writes are outside this Phase 1 implementation. The report and code were developed with Codex assistance; results and limitations are explicitly recorded.

[1] LLM4Ops, Cloud-OpsBench: github.com/LLM4Ops/Cloud-OpsBench (MIT). [2] CSE291A Fall 2026, Option 5 project specification and course project page: cseweb.ucsd.edu/~yiying/cse291a-fall26/project/.

Appendix: timeline and reproducibility

This appendix is separate from the two-page Phase 1 writeup.

Window	Milestone	Evidence / completion condition
October 8	Phase 1 local package	Workflow, fixed interventions, measured baselines, software controls, raw traces and this report
Before October 28	Author review and submission	Review every claim and course updates; submit code/data and report with this timeline
October 29 - November 15	Recovery and learning evaluation	Shared-state/dense faults, semantic verification, missing provenance; independent memory gates
November 16 - November 29	Scheduling and interaction study	Frozen loads and seeds; response/recovery tradeoffs, overload and component ablations
November 30 - December 10	Final replication and report	Freeze final configuration; run held-out cases; 4-6 page final PDF, code and limitations

Local artifact inventory

PHASE1_README.txt describes exact offline reproduction commands and live-run requirements. MANIFEST.sha256.json authenticates bundled bytes against the recorded index (integrity check, not a trusted signature). Source, tests, predeclared configs, environment lock, reports, generated metrics and raw model requests/responses are included. The offline vendor subset contains the native evaluator, the three case labels and the MIT license. No credentials or paid-account setup are bundled.

Offline checks regenerate controls, recalculate native scores from saved traces, reproduce matrix aggregates, and run the available unit tests. Snapshot execution tests require a separate pinned benchmark checkout and are explicitly skipped when it is absent. A fresh live replay additionally requires the benchmark checkout and an authenticated Codex installation; outputs are stochastic and consume subscription quota.

Interpretation of deliverables

Simulation outputs and live-model evidence are separate. Operational writes never occur. Native scores are accompanied by source-fidelity and semantic review because a correct root label can coexist with false explanatory facts. Raw inference events preserve rejected outputs and actual usage. The submission package is prepared locally; no course-platform submission is asserted.